

Federal University of Ceará
Department of Electrical Engineering
Laboratory of Electronic Systems and Circuits (LESC)

FPGA Accelerated Aging Environment

Student Onboarding Guide

A complete reference for understanding, setting up, and operating
the hardware-in-the-loop burn-in experiment platform.

Revision: May 2026

Contents

1	Introduction	4
1.1	What the project does	4
1.2	Who this document is for	4
1.3	Document scope	4
2	Scientific Background	6
2.1	Why FPGAs age	6
2.2	Negative Bias Temperature Instability (NBTI)	6
2.2.1	Microscopic mechanism: interface trap generation	6
2.2.2	Reaction-Diffusion model	6
2.2.3	Recovery	7
2.2.4	Temperature acceleration	7
2.3	Hot Carrier Injection (HCI)	7
2.4	How degradation manifests in digital timing	8
2.5	Why FPGAs are useful for aging experiments	8
3	System Overview	9
3.1	Architecture block diagram	9
3.2	Hardware bill of materials	9
3.3	Experiment overview	10
4	FPGA RTL Architecture	11
4.1	Target device	11
4.2	Module hierarchy	11
4.3	Clock domains	11
4.4	The aging-sensitive critical path	12
4.4.1	Fixed placement and routing	12
4.5	The metastability sensor (<code>modern_sensible</code>)	13
4.6	Dual-adder design (<code>adder_canary</code>)	13
4.6.1	Sensor adder (<code>u_sensor</code>): timing measurement	13
4.6.2	Canary adder (<code>u_canary</code>): functional failure detection	14
4.7	The phase-sweep FSM (<code>controller_controller</code>)	14
4.7.1	Sticky alarm latch	15
4.8	XADC interface (<code>temp_catcher</code>)	15
4.8.1	Conversion formulas	15
4.9	UART interface	16
4.9.1	Trigger (PC → FPGA)	16
4.9.2	15-byte response packet (FPGA → PC)	16
4.10	Building the bitstream	16
5	Software Architecture	17
5.1	Applications overview	17

5.2	Launching the application	17
5.3	Worker architecture (QThread pattern)	17
5.4	DUTWorker: reading FPGA data	18
5.5	ArduinoWorker	18
5.6	PSUWorker (IT6502D)	18
5.7	Configuration system	19
5.8	Data logger	19
5.9	GUI layout	20
6	Communication Protocols	21
6.1	DUT protocol (FPGA)	21
6.2	Arduino protocol	21
6.3	PSU protocol (IT6502D)	21
7	Control Loops	22
7.1	Oven temperature control (PID)	22
7.1.1	Plant identification	22
7.1.2	SIMC tuning	22
7.1.3	Implementation	22
7.2	VCCINT closed-loop voltage trim	23
7.3	DUT outer temperature loop	23
8	Setting Up a Fresh Workstation	24
8.1	System requirements	24
8.2	Step 1 — Install system dependencies	24
8.3	Step 2 — Clone the repository	24
8.4	Step 3 — Install Vivado (if needed)	24
8.5	Step 4 — Configure udev rules for serial and JTAG devices	25
8.5.1	Add user to the dialout group	25
8.5.2	Install Digilent USB cable rules	25
8.6	Step 5 — Identify serial port assignments	25
8.7	Step 6 — Identify the PSU VISA resource	26
8.8	Step 7 — First run	26
9	Running an Experiment	27
9.1	Pre-experiment checklist	27
9.2	Setup Dialog	27
9.3	Setting experiment parameters	27
9.4	Starting the test	28
9.5	Monitoring	28
9.6	Safety limits	28
9.7	Stopping the test	28
10	Data Analysis	29
10.1	CSV file structure	29
10.2	Loading data in Python (Pandas)	29
10.3	Typical result signature	29
10.4	Fitting the degradation model	30

11 Other Applications	31
11.1 App_2Nexys (dual DUT)	31
11.2 App_FPGAging_Slack_Sensor (UltraScale+ / SBCCI target)	31
12 Troubleshooting	32
12.1 Serial port not accessible	32
12.2 Wrong ttyUSB port selected (JTAG vs UART)	32
12.3 App_2Nexys fails to program FPGAs	32
12.4 PSU VISA string error	32
12.5 dut_slack shows erratic jumps	33
12.6 Vivado timing reports show negative slack on the sensor path	33
A FOPDT Step Test Procedure	34
B Regenerating Fixed PnR Constraints	35

1. Introduction

This document describes the **FPGA Accelerated Aging Environment** developed at LESC (Laboratory of Electronic Systems and Circuits, Federal University of Ceará). Its purpose is to give a new student or contributor everything needed to understand the scientific motivation, operate the equipment, and extend the software or RTL.

1.1 What the project does

Semiconductor transistors degrade over time when subjected to electrical stress. The key mechanisms — Negative Bias Temperature Instability (NBTI) and Hot Carrier Injection (HCI) — cause the threshold voltage V_{th} of MOSFET devices to drift, increasing propagation delays in logic gates. If degradation is severe enough, a once-working digital circuit will begin to fail timing constraints and produce erroneous outputs.

Studying these phenomena experimentally requires either very long stress times at nominal conditions, or *accelerated aging*: raising the temperature and/or supply voltage to stress the device much faster. This project implements a complete hardware-in-the-loop environment for accelerated aging of commercial FPGAs.

The *device under test* (DUT) is a Digilent Nexys4 DDR board populated with a Xilinx Artix-7 FPGA (xc7a100tcsg324-1). It is placed inside a temperature-controlled oven, powered by a programmable benchtop supply, and continuously monitored by a custom RTL bitstream that measures timing degradation in real time using a metastability-based phase sensor. All measurements are streamed over UART to a host PC where a Python application logs timestamped CSV files and displays live plots.

1.2 Who this document is for

This guide assumes the reader has:

- Familiarity with digital design concepts (clock domains, setup/hold time, FPGAs).
- Basic SystemVerilog or Verilog reading ability.
- Experience with Python and comfort at the Linux command line.
- Some exposure to control systems (PID loops, transfer functions).

No prior knowledge of the project is required.

1.3 Document scope

Chapter 2 covers the device-physics background — NBTI, HCI, and the reaction-diffusion model — at a graduate level. Chapter 3 gives a system-level overview. Chapters 4 and 5 describe the FPGA RTL and the Python software in depth. Chapter 6 documents all

communication protocols. Chapter 7 explains the control loops. Chapter 8 provides step-by-step workstation setup instructions. Chapters 9 and 10 cover running an experiment and analyzing the results.

2. Scientific Background

2.1 Why FPGAs age

A modern FPGA is built from hundreds of thousands of CMOS transistors. Each transistor consists of a gate electrode separated from the silicon channel by a thin gate oxide. The electric field across this oxide, combined with the kinetic energy of charge carriers, gradually damages the oxide and the Si/SiO₂ interface. The primary mechanisms that affect digital timing are NBTI and HCI.

2.2 Negative Bias Temperature Instability (NBTI)

NBTI is the dominant aging mechanism in modern sub-micron CMOS. It predominantly affects **PMOS** transistors biased in inversion (gate-to-source voltage $V_{GS} < 0$), which is precisely the stressed condition during logic ‘1’ propagation through a PMOS pull-up.

2.2.1 Microscopic mechanism: interface trap generation

The Si/SiO₂ interface is chemically passivated by hydrogen. Silicon dangling bonds — known as P_b centres — are tied up as Si-H bonds at the interface. Under a strong negative gate bias and elevated temperature, these bonds dissociate:



The resulting silicon dangling bond becomes a positively charged interface trap that captures a hole and shifts the PMOS threshold voltage:

$$\Delta V_{th} = \frac{q N_{it}}{C_{ox}} \quad (2.2)$$

where N_{it} is the interface trap density, q is the electron charge, and C_{ox} is the oxide capacitance per unit area. In addition, some hydrogen and holes are captured as *oxide traps* in the bulk SiO₂, contributing a further positive charge that also increases $|V_{th}|$.

An increased $|V_{th}|$ weakens the on-current of the PMOS ($I_D \propto V_{GS} - V_{th}$), reducing its drive strength and increasing the propagation delay of every gate where a PMOS pull-up is on the critical path.

2.2.2 Reaction-Diffusion model

The temporal evolution of ΔV_{th} is described by the Reaction-Diffusion (R-D) model. During the **stress phase**, interface traps are generated at the interface (reaction) and hydrogen diffuses away into the oxide (diffusion), preventing back-passivation. The rate-limiting step

is the diffusion of atomic or molecular hydrogen. Solving the coupled reaction-diffusion equations yields the well-known power-law time dependence:

$$\Delta V_{th}(t) = A \left(\frac{V_{GS, stress}}{t_{ox}} \right)^m \exp\left(\frac{-E_a}{k_B T}\right) \cdot t^n \quad (2.3)$$

where:

- $n \approx 0.1\text{--}0.25$ is the time exponent (determined by whether H or H₂ diffuses);
- $E_a \approx 0.1\text{--}0.2$ eV is the activation energy for the diffusion process;
- k_B is Boltzmann's constant and T is absolute temperature;
- m is the field-acceleration exponent;
- A is a technology-dependent pre-factor.

2.2.3 Recovery

A key (and sometimes inconvenient) feature of NBTI is that degradation is *partially reversible*. When the gate bias is removed, residual hydrogen at the interface back-passivates some of the dangling bonds: $\text{Si} \bullet + \text{H} \rightarrow \text{Si-H}$. This **recovery** can mask the true degradation if measurements are taken after a rest period. In the long run, however, the degradation is not fully recovered — the component that does not recover (often called the permanent component) accumulates monotonically and represents the true aging damage.

In this experiment the DUT is stressed *continuously* (no recovery periods), so the measured ΔV_{th} is representative of the full stress-only trajectory.

2.2.4 Temperature acceleration

The Arrhenius relationship gives the ratio of experiment lifetimes at two temperatures:

$$\text{AF} = \frac{\tau_{use}}{\tau_{stress}} = \exp\left[\frac{E_a}{k_B} \left(\frac{1}{T_{use}} - \frac{1}{T_{stress}} \right)\right] \quad (2.4)$$

For $E_a = 0.15$ eV, stressing at 125 °C instead of 25 °C gives an acceleration factor of roughly 8×. For higher activation energies the acceleration is even more pronounced. This is why the experiment uses an oven: a DUT at 125 °C sees months of aging compressed into days.

2.3 Hot Carrier Injection (HCI)

HCI predominantly affects NMOS transistors operating at high drain-to-source voltage V_{DS} . Near the drain, the lateral electric field $\mathcal{E} \approx V_{DS}/L$ is highest. Electrons accelerating towards the drain can acquire kinetic energies well above the thermal average (“hot” carriers). Some fraction of these hot carriers have sufficient energy to surmount the Si/SiO₂ barrier (≈ 3.1 eV) and be injected into the gate oxide, where they are trapped or generate interface states.

The resulting threshold voltage shift and transconductance degradation slow NMOS pull-down paths, directly increasing fall times in logic gates. HCI is worst at intermediate V_{GS} values that maximise the substrate current (maximum multiplication condition), and it accelerates strongly with supply voltage.

In this experiment, the supply voltage V_{CCINT} is elevated above the nominal 1.0 V to accelerate both NBTI and HCI simultaneously. The trade-off is that the acceleration factor must be chosen carefully: too high a voltage can trigger catastrophic dielectric breakdown rather than controlled, gradual aging.

2.4 How degradation manifests in digital timing

From a circuit perspective, NBTI and HCI both increase propagation delay t_{pd} along paths that traverse affected transistors. In a synchronous design, the critical constraint is the *setup time* relation:

$$t_{pd} \leq T_{clk} - t_{setup} - t_{skew} \quad (2.5)$$

The *timing slack* is the margin available:

$$\text{slack} = T_{clk} - t_{setup} - t_{skew} - t_{pd} \quad (2.6)$$

As t_{pd} increases due to aging, slack decreases. When slack reaches zero, the flip-flop at the path endpoint can no longer reliably capture the correct data value, and functional errors begin to occur.

This project measures timing degradation by directly quantifying this slack using an on-chip phase-sensitive sensor, described in detail in Chapter 4.

2.5 Why FPGAs are useful for aging experiments

Commercial FPGAs offer several advantages as aging test vehicles:

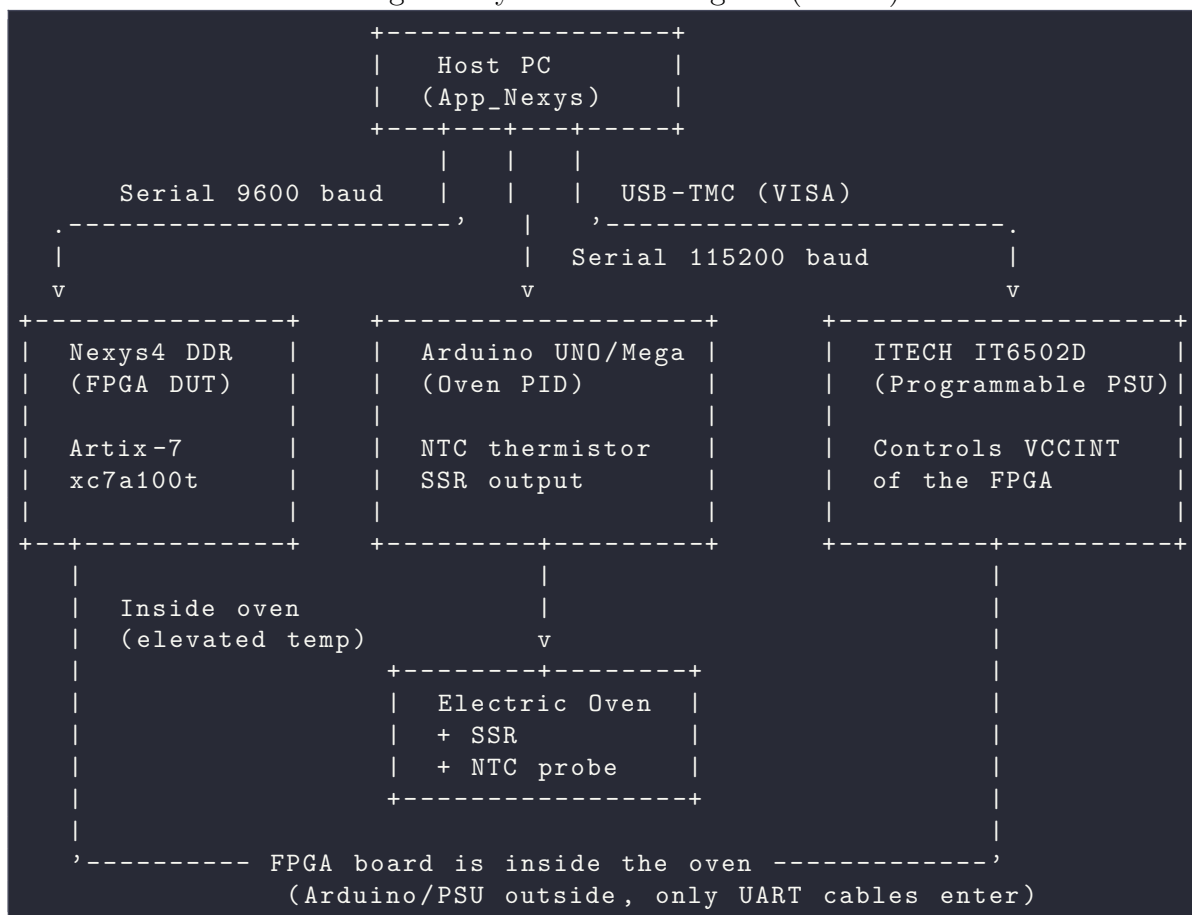
1. **Fixed routing:** the FPGA's routing fabric is programmed once (fixed PnR), so the same physical transistors are stressed repeatedly. Unlike an ASIC, there is no process-to-process variation between samples.
2. **Reprogrammability:** the bitstream can be changed to instrument different paths or add new sensors without re-fabricating silicon.
3. **Built-in XADC:** Xilinx FPGAs include an on-chip Analogue-to-Digital Converter (XADC) that continuously measures die temperature and supply voltages without external probes.
4. **MMCM phase control:** the Mixed-Mode Clock Manager (MMCM) supports dynamic phase shift via the DRP interface, enabling sub-nanosecond timing manipulation at runtime.

3. System Overview

3.1 Architecture block diagram

The complete system is shown below. Three separate subsystems communicate with the host PC:

Listing 3.1: System block diagram (ASCII)



Note

The Arduino and PSU are optional. The app can run with only the FPGA connected (no temperature control, no programmable supply). However, meaningful aging experiments require all three subsystems.

3.2 Hardware bill of materials

Component	Role	Interface
Digilent Nexys4 DDR	DUT (Artix-7 FPGA)	USB-UART (FTDI)

Component	Role	Interface
Electric oven / incubator	Thermal stress chamber	—
Solid-State Relay (SSR)	Oven power switching	Digital GPIO (Arduino)
NTC thermistor (10 k Ω)	Oven temperature sensing	Analogue input (Arduino)
Arduino UNO or Mega	PID controller	USB-UART, 115200 baud
ITECH IT6502D	Programmable DC supply	USB-TMC (VISA)
Host PC (Linux)	Data acquisition & control	USB hub

3.3 Experiment overview

A typical aging experiment proceeds as follows:

1. The FPGA board is placed inside the oven. USB/UART cables pass through a small opening in the oven door.
2. The IT6502D PSU is connected to the FPGA’s external power header to supply an elevated V_{CCINT} .
3. The host PC is powered on; all USB devices enumerate.
4. The operator launches the **App_Nexys** application and configures the serial ports in the Setup Dialog.
5. The operator sets the oven temperature setpoint (e.g., 125 °C) and the target V_{CCINT} (e.g., 1.1 V).
6. The operator clicks **Start Test**. The application enters autonomous mode:
 - The Arduino PID loop ramps the oven to the setpoint.
 - The PSU delivers the commanded voltage; the closed-loop trim adjusts it to match the XADC-measured V_{CCINT} .
 - Every second, the PC sends a trigger byte (0x54) to the FPGA.
 - The FPGA responds with a 15-byte packet: die temperature, slack counter, V_{CCINT} , failure flag, and canary metrics.
 - All data is appended to a timestamped CSV file.
7. The experiment runs unattended for one to seven days.
8. The operator clicks **Stop Test** and retrieves the CSV for analysis.

4. FPGA RTL Architecture

4.1 Target device

Board: Digilent Nexys4 DDR
Device: Xilinx Artix-7 xc7a100tcsg324-1
Top module: nexys4_aging_top
Source language: SystemVerilog (pure RTL, no block design)
Toolchain: Vivado 2025.2+

4.2 Module hierarchy

Listing 4.1: RTL module hierarchy

```
nexys4_aging_top      (src/rtl/top/)
  clk_wiz_0           MMCM -- dynamic phase shift
    (src/ip/)
  XADC                on-chip ADC (primitive)
  temp_catcher         XADC DRP monitor (aging_sensor/)
  modern_sensible      3-FF metastability sensor
    (aging_sensor/)
  [2-FF synchronizer] alarm CDC from clk_en to clk_sys
  adder_canary         dual-adder sensor (aging_sensor/)
    u_sensor (ripple_adder)    toggle-driven, timing
    measurement
    u_canary (ripple_adder)    counter-driven, functional
    canary
  controller_controller phase-sweep FSM (aging_sensor/)
  failure_holder       sticky failure latch
    (aging_sensor/)
  uart_rx              PC -> FPGA trigger receiver
    (uart/)
  sensor_stream        FPGA -> PC packet serialiser
    (uart/)
  uart_tx              UART transmitter (uart/)
  BINtoBCD (x2)        for 7-segment display (display/)
  DisplayController    8-digit 7-seg multiplexer
    (display/)
  vio_0                Xilinx VIO debug core
```

4.3 Clock domains

All three clocks are generated from a single MMCM (`clk_wiz_0`) driven by the 100 MHz board oscillator. Using one MMCM guarantees that all clocks are phase-coherent and that dynamic phase shifts affect only the selected output.

Clock	Freq.	Phase	Drives
clk_sys	100 MHz	0° (fixed)	All RTL logic; FF2 of sensor
psclk	100 MHz	0° + dynamic shift	FF1 of sensor (<code>modern_sensible</code>)
clk_en	100 MHz	100° (fixed)	FF3 (alarm latch) of sensor

The `alarm_sig` output of `modern_sensible` is in the `clk_en` domain. It is passed through a standard 2-FF synchroniser (`alarm_meta` / `alarm_sync`) before being used by any `clk_sys` logic.

4.4 The aging-sensitive critical path

The path whose degradation we want to measure is a 16-bit ripple-carry adder (`ripple_adder`) built entirely from LUT primitives:

Listing 4.2: LUT full adder instantiation (`lut_full_adder.sv`)

```
(* DONT_TOUCH = "yes" *)
LUT2_L #(.INIT(4'b0110)) XOR_SUM (.LO(s_wire), .IO(a_i), .I1(b_i));
(* DONT_TOUCH = "yes" *)
LUT3 #(.INIT(8'hE8)) AND_MAJ (.O(c_o), .IO(a_i), .I1(b_i), .I2(c_i));
```

The `DONT_TOUCH` attribute is *critical*. Without it, Vivado’s synthesiser replaces the LUT ripple chain with the device’s dedicated `CARRY4` primitives, which:

1. Use a completely different physical path (carry chain routing, not LUT routing).
2. Are not constrained by the fixed PnR constraints.
3. Age at a different rate from the LUT fabric.

The result would be that the measured timing degradation does not reflect LUT aging.

4.4.1 Fixed placement and routing

The RTL project uses *fixed placement and routing* (`fixed_pnr_constraints.xdc`). This file pins every LUT and flip-flop of the sensor to specific BEL locations and locks the routing. The effect is:

- All repeated experiments use exactly the same physical transistors.
- There is no run-to-run variation in timing due to the place-and-route algorithm.
- Measurements from different experimental sessions are directly comparable.

Warning

Never modify the `LOC`, `BEL`, or `FIXED_ROUTE` constraints in `fixed_pnr_constraints.xdc` without understanding the consequences. Any change invalidates all previously collected data. Regenerate from a new checkpoint only after a deliberate PnR run (see Section 4.10).

4.5 The metastability sensor (`modern_sensible`)

The sensor detects when the critical path output (`crit_bit`) is arriving at a flip-flop too close to the clock edge, i.e., when timing slack approaches zero.

Listing 4.3: `modern_sensible.sv` (simplified)

```
// FF1: captures crit_bit on the PHASE-SHIFTED clock (psclk)
FDCE FF1 (.Q(ff1), .C(psclk), .D(in_sensor), ...);

// FF2: captures crit_bit on the REFERENCE clock (sclk = clk_sys)
FDCE FF2 (.Q(ff2), .C(sclk), .D(in_sensor), ...);

// XOR: detects disagreement between the two captures
LUT2_L #(.INIT(4'b0110)) XOR1 (.LO(xor_out), .I0(ff1), .I1(ff2));

// FF3: latches the XOR result on a third clock (clk_en, 100 deg
//      offset)
FDCE FF3 (.Q(alarm), .C(clk_en), .D(xor_out), ...);
```

The operating principle is:

1. FF2 always captures `crit_bit` reliably on `clk_sys`. It represents the “ground truth” value.
2. FF1 captures `crit_bit` on `psclk`, which is phase-shifted relative to `clk_sys` by a controllable amount.
3. When `psclk` is close in phase to `clk_sys`, FF1 and FF2 see the same signal edge and capture the same value $\Rightarrow \text{XOR} = 0 \Rightarrow$ no alarm.
4. As the phase of `psclk` is shifted (decremented), a point is reached where FF1’s setup window straddles the transition of `crit_bit`. FF1 captures the *old* value while FF2 captures the *new* value $\Rightarrow \text{XOR} = 1 \Rightarrow$ alarm fires.
5. FF3 latches the alarm on `clk_en` (100° offset) to avoid hold-time hazards in the XOR output.

The number of MMCM phase-shift steps at which the alarm fires is the timing margin measurement. This is the **slack** value (`display_value`).

4.6 Dual-adder design (`adder_canary`)

Two structurally identical ripple-adder instances serve different roles:

4.6.1 Sensor adder (`u_sensor`): timing measurement

The sensor adder must exercise the full carry chain on every clock cycle to give a consistent, repeatable timing stimulus. A free-running counter would mostly add small numbers that do not propagate carry through all 16 stages. Instead, a `toggle` flip-flop alternates `a_sensor` between two boundary values:

$$\mathbf{a_sensor} = \begin{cases} 16\text{h}5555 & \text{if toggle} = 0 \\ 16\text{h}5556 & \text{if toggle} = 1 \end{cases} \quad (4.1)$$

$$\mathbf{b} = 16\text{hAAAA} \quad (\text{constant}) \quad (4.2)$$

With these values, `sum_sensor[15]` transitions on *every* clock cycle because the boundary values straddle the carry-out threshold of every stage. This forces carry propagation through all 16 LUT stages on each cycle, making the timing stimulus deterministic and maximally stressful on the critical path.

4.6.2 Canary adder (`u_canary`): functional failure detection

The canary adder uses a free-running counter input `a` and the same constant `b = 0xAAAA`. Its output is compared against a reference computation (`sum_ref`) implemented in standard synthesis (may use `CARRY4`). Because both adders use identical transistors (same placement), the canary detects when aging has progressed far enough to cause *functional errors*:

- **wrong**: the canary adder output at the moment of the last alarm.
- **correct**: the reference adder output at the same moment.
- **error_count**: running count of cycles where the two outputs differ (saturates at 0xFFFF).
- **error_any**: sticky flag, set on the first ever mismatch.

Note

The canary metrics are logged to CSV but not displayed in the GUI. They are useful for post-processing to correlate the onset of timing degradation with eventual functional failure.

4.7 The phase-sweep FSM (`controller_controller`)

The FSM runs autonomously and continuously. It performs a complete measurement sweep every few milliseconds without any trigger from the host PC.

Listing 4.4: FSM state diagram

```

CHECK_ALARM --> alarm?  --yes--> DONE
                        --no --> INIT_SHIFT
                        |
                        (decrement psclk phase by one step)
                        v
                        WAIT_SHIFT
                        |
                        '----(inc_count++)-----'
DONE --> latch display_value = inc_count
      --> RESET_PHASE

```

```

RESET_PHASE --> reset_count >= inc_count? --yes--> IDLE
                                         --no --> WAIT_RESET
                                         |
                                         (increment psclk phase, back to 0) |
                                         <--(reset_count++)-----'

IDLE --> clear inc_count, reset_count
      --> CHECK_ALARM (immediately restarts next sweep)

```

Key insight: as the device ages, `crit_bit` transitions later relative to the clock edge (path delay increases). The alarm therefore fires after *fewer* phase-shift steps. Thus `display_value` decreases monotonically as the device degrades. A plot of `dut_slack` vs. time showing a downward trend is the primary evidence of aging.

4.7.1 Sticky alarm latch

Because `crit_bit` toggles every clock cycle, the `alarm` signal from `modern_sensible` is a brief 1–2 cycle pulse rather than a steady level. `CHECK_ALARM` samples `alarm` for exactly one cycle and would frequently miss this pulse. A sticky latch captures the first alarm of each sweep and holds it until `IDLE` clears it, ensuring the FSM never misses the alarm regardless of pulse timing.

4.8 XADC interface (`temp_catcher`)

The Xilinx XADC monitors two internal signals:

- **Temperature** (register 0x00): die temperature using an on-chip bandgap diode.
- **VCCINT** (register 0x01): internal core supply voltage.

The `temp_catcher` module polls these registers via the DRP (Dynamic Reconfiguration Port) interface. A FSM asserts `DEN` (data enable), waits for `DRDY` (data ready), reads `D0`, then repeats for the next register. A 128-cycle watchdog retries if `DRDY` does not arrive (which can happen if `DEN` is pulsed incorrectly).

4.8.1 Conversion formulas

The 12-bit ADC code (bits [15:4] of the 16-bit register) is converted to physical units by `temp_catcher` in fixed-point arithmetic:

$$T \text{ [m}^\circ\text{C]} = \frac{\text{ADC_code}[15:4] \times 503975}{2^{12}} - 273150 \quad (4.3)$$

$$V_{CCINT} \text{ [mV]} = \frac{\text{ADC_code}[15:4] \times 3000}{2^{12}} \quad (4.4)$$

These values are transmitted in the serial packet as 24-bit integers (millidegrees and millivolts). The Python application divides by 1000 to recover floating-point Celsius and volts.

4.9 UART interface

4.9.1 Trigger (PC → FPGA)

The PC sends the single byte 0x54 (ASCII ‘T’). The `uart_rx` module detects this byte and asserts the `sendin` signal, which causes `sensor_stream` to latch all sensor values and begin transmitting the 15-byte response.

4.9.2 15-byte response packet (FPGA → PC)

All fields are Little Endian (LSB first). The UART runs at 9600 baud.

Byte(s)	Field	Width	Description
0–2	<code>temp</code>	24-bit	Die temperature (m°C), divide by 1000 for °C
3–4	<code>sensor</code>	16-bit	Phase-sweep step count (slack)
5–7	<code>vccint</code>	24-bit	VCCINT (mV), divide by 1000 for V
8	<code>failure</code>	8-bit	Bit 0: functional failure latch
9–10	<code>wrong</code>	16-bit	Canary: last wrong adder result
11–12	<code>correct</code>	16-bit	Canary: expected result at alarm time
13–14	<code>error_count</code>	16-bit	Canary: cumulative mismatch count

Warning

The byte layout above is consumed verbatim by `DUTWorker.poll_data()` in the Python application. Never reorder fields or change widths in `sensor_stream.sv` without updating the parser.

4.10 Building the bitstream

Listing 4.5: Build commands (from repo root)

```
cd vivado/aging_study_nexys4ddr

# Sanity check (no Vivado needed)
scripts/check_layout.sh

# Create Vivado project (.xpr)
scripts/create_project.sh          # batch mode
scripts/create_project.sh --gui    # open in Vivado GUI

# Build bitstream (creates artifacts/aging_study_nexys4ddr.bit)
scripts/build_bitstream.sh --jobs 8

# Regenerate fixed PnR constraints from a new checkpoint
scripts/build_bitstream.sh --jobs 8 --refresh-ref

# Remove build output
scripts/clean.sh
```

The bitstream and timing reports land in `artifacts/`. The pre-built bitstream used for App_2Nexys is in `vivado/aging_study_nexys4ddr/bitstreams/`.

5. Software Architecture

5.1 Applications overview

There are three Python GUI applications. This guide focuses on **App_Nexys** (single DUT, Artix-7 Nexys4 DDR), which is the primary entry point for new students. The other two are briefly described in Chapter 11.

All apps share the same software patterns: a Qt main window, QThread-based workers, a global configuration module, and a CSV data logger.

5.2 Launching the application

Listing 5.1: Running App_Nexys

```
# Recommended: root launcher (choose at the dialog)
./run.sh

# Direct launch
cd App_Nexys && ./run.sh
```

The `run.sh` script:

1. Creates `.venv/` using the system Python 3 if it does not exist.
2. Installs dependencies from `requirements.txt`.
3. Sets `LD_LIBRARY_PATH` to prefer the `venv`'s Qt libraries over system packages (resolves Qt version conflicts on Linux Mint).
4. Executes `python App.py`.

5.3 Worker architecture (QThread pattern)

The GUI event loop must remain responsive at all times. All hardware communication runs in background QThreads. Each worker is a QObject with a QTimer driving its polling loop. Signals carry data back to the main thread.

Listing 5.2: Thread structure

```
Main Thread (Qt event loop)
|
+-- ArduinoWorker (QThread)      -- polls Arduino at ~200 ms
    intervals
    |   QTimer --> poll_data()
    |   Signal: arduino_data_received(dict)
    |
+-- PSUWorker (QThread)          -- polls IT6502D at ~500 ms
    intervals
```

```

|         QTimer --> poll_data()
|         Signal: psu_data_received(dict)
|
+-- DUTWorker (QThread)          -- sends 'T', reads 15-byte packet
    at ~1s
|         QTimer --> poll_data()
|         Signal: dut_data_received(dict)
|
+-- TestSequencer (QThread)      -- orchestrates, writes CSV, runs
    safety checks
    QTimer --> log_data_tick() (1 Hz)
    Receives signals from all three workers
    Runs VCCINT closed-loop trim
    Runs DUT outer temperature loop (every 1800 ticks)

```

5.4 DUTWorker: reading FPGA data

On each poll tick, DUTWorker:

1. Flushes the serial receive buffer.
2. Sends the byte `b'\x54'` (`0x54 = 'T'`).
3. Reads exactly 15 bytes.
4. Unpacks them according to the packet layout (Table in Section 4).
5. Converts raw values: `temp_raw / 1000.0 → dut_temp_c`; `vccint_raw / 1000.0 → dut_volt`.
6. Emits `dut_data_received` with a dictionary containing all fields.

Boot transients (`temp > 200 °C` or `vccint > 2.5 V`) are discarded to avoid spurious data while the XADC is settling after power-on.

5.5 ArduinoWorker

The Arduino communicates via ASCII lines over UART at 115200 baud.

- **PC → Arduino:** `SET_SETPOINT,<value>\n`
- **Arduino → PC:** `DATA,<temp>,<sp>,<out>\n` where `out` is the PID output (0–100%).

5.6 PSUWorker (IT6502D)

The ITECH IT6502D is accessed via PyVISA with the `@py` (`pyvisa-py`) backend. The VISA resource string must be a USB resource of the form:

```
USB0::0x1AB1::0x0E11::IT6502D300004::INSTR
```

To discover the correct string on a given machine:

```
python3 -c "import pyvisa; rm=pyvisa.ResourceManager('@py');
print(rm.list_resources())"
```

SCPI commands used:

- MEAS:VOLT? — read output voltage.
- MEAS:CURRE? — read output current.
- VOLT <value> — set output voltage.
- OUTP ON / OUTP OFF — enable/disable output.

5.7 Configuration system

`config.py` is a global-mutable-state module. All workers import it directly; no dependency injection is used. Settings are persisted to `settings.json` in the app's working directory.

Key configuration values:

Parameter	Default	Meaning
DUT_BAUD	9600	FPGA UART baud rate
ARDUINO_BAUD	115200	Arduino UART baud rate
PID_KP	2.78	Oven PID proportional gain
PID_KI	0.00106	Oven PID integral gain
PID_KD	5.0	Oven PID derivative gain
VCCINT_SETPOINT_V	1.0 V	Target FPGA core voltage
VOLTAGE_KP	0.1 V/V	VCCINT P-only trim gain
MAX_OVEN_TEMP_C	130 °C	Safety: oven hard limit
MAX_DUT_TEMP_C	160 °C	Safety: DUT hard limit
MAX_PSU_CURRENT_A	3.0 A	Safety: PSU current limit

5.8 Data logger

`DataLogger` (in `logger.py`) writes a single CSV file per test run. Files are named `<test_name>_<YYYYMMDD_HHMMSS>.csv` and saved to `App_Nexys/test_logs/`.

Each file begins with a comment block recording the PID parameters, FOPDT model coefficients, safety limits, and the start timestamp. The data section contains one row per second with the following columns:

Column	Description
<code>time_sec</code>	Elapsed seconds since test start
<code>oven_temp_c</code>	Oven air temperature (°C)
<code>oven_setpoint_c</code>	Current oven setpoint (°C)
<code>oven_output_pct</code>	PID output to SSR (0–100%)
<code>psu_voltage_v</code>	PSU output voltage (V), measured by PSU
<code>psu_current_a</code>	PSU output current (A)

Column	Description
dut_temp_c	FPGA die temperature (°C, from XADC)
dut_slack	Phase-sweep step count (timing margin)
dut_volt	FPGA VCCINT (V, from XADC)

5.9 GUI layout

The main window uses the Catppuccin Mocha dark theme.

Listing 5.3: Window layout

```
+-----+
| [Oven Control + PID info]  [PSU]  [Test Controls]          | <-
| top bar                                                         |
+-----+
| Tabs:                                                            |
| [Sensor]  [Temperature]  [Voltage]  [Log]                    |
+-----+
| (QSplitter)                                                      |
| Tab content area                                                |
| ----- |
| Event log (always visible, pinned below tabs)                  |
+-----+
| Status bar: DUT [connected]  Arduino [connected]  PSU [OK]    |
+-----+
```

6. Communication Protocols

6.1 DUT protocol (FPGA)

- **Baud rate:** 9600, 8N1.
- **Trigger:** PC sends single byte 0x54 (ASCII ‘T’).
- **Response:** FPGA sends 15 bytes, Little Endian, as described in Section 4.
- **Timing:** response begins within one UART character time (about 1 ms at 9600 baud) after the trigger is received.

6.2 Arduino protocol

- **Baud rate:** 115200, 8N1.
- **PC → Arduino:** SET_SETPOINT,<float>\n (e.g. SET_SETPOINT,125.0\n).
- **Arduino → PC:** DATA,<temp>,<setpoint>,<output>\n (e.g. DATA,123.45,125.00,47.30\n). Sent autonomously at the PID sample interval (5000 ms by default).

6.3 PSU protocol (IT6502D)

- **Physical layer:** USB-TMC (USB Test and Measurement Class). No baud rate — full USB speed.
- **Library:** PyVISA with @py backend (pyvisa-py + libusb).
- **Commands:** SCPI (Standard Commands for Programmable Instruments).

Warning

The PSU VISA resource string *must* start with USB:: or USB0::. Legacy serial paths (/dev/ttyUSB*) or ASRL strings are rejected at startup by `config.load_config()` to prevent a “Could not configure port” error.

7. Control Loops

7.1 Oven temperature control (PID)

7.1.1 Plant identification

The oven was modelled as a First-Order Plus Dead Time (FOPDT) system using a step-test experiment (`Arduino-ESP/FOPDT_Step_Test.ino`):

$$G(s) = \frac{1.56 e^{-150.6s}}{1307.2s + 1} \quad (7.1)$$

Parameters:

- $K = 1.56^\circ\text{C}/\%$ (steady-state gain)
- $\theta = 150.6\text{ s}$ (dead time)
- $\tau = 1307.2\text{ s}$ (time constant)
- $\theta/\tau = 0.115$ (process difficulty ratio)

7.1.2 SIMC tuning

PID parameters were computed with the SIMC (Skogestad Internal Model Control) method, setting the desired closed-loop time constant $\tau_c = \theta$:

$$K_p = \frac{1}{K} \cdot \frac{\tau}{\tau_c + \theta} = 2.78 \quad (7.2)$$

$$K_i = \frac{K_p}{\min(\tau, 4(\tau_c + \theta))} = 0.00106\text{ s}^{-1} \quad (7.3)$$

$$K_d = K_p \cdot \frac{\theta}{2} = 5.0\text{ s} \quad (7.4)$$

Warning

Do not change these PID values without conducting a new FOPDT step test. The parameters are finely tuned to the specific oven and SSR combination in the lab. Wrong values will cause sustained oscillation or sluggish response.

7.1.3 Implementation

The PID runs on the Arduino at a sample interval of 5000 ms using the ArduPID 1.0.1 library. Key API calls:

Listing 7.1: ArduPID 1.0.1 usage

```
pid.setTunings(Kp, Ki, Kd); // replaces begin() from v0.2.1
pid.setILimits(-100, 100); // replaces setWindUpLimits()
pid.setDtMs(5000); // replaces setSampleTime()
output = pid.compute(input); // takes input; returns output
```

The Arduino sends the PID output as a duty cycle percentage to the SSR (Solid-State Relay), which switches mains power to the oven heating element.

7.2 VCCINT closed-loop voltage trim

The FPGA’s internal supply V_{CCINT} is measured by the XADC and included in every 15-byte packet (`dut_volt`). The TestSequencer applies a P-only correction every log tick (1 Hz):

$$v_{cmd}(k) = v_{cmd}(k-1) + K_{p,V} \cdot (V_{set} - V_{meas}(k)) \quad (7.5)$$

with $K_{p,V} = 0.1 \text{ V/V}$. The command is clamped to $[0.0 \text{ V}, 1.5 \text{ V}]$. This compensates for cable resistance and the PSU’s load regulation, ensuring the FPGA core actually operates at the commanded voltage.

7.3 DUT outer temperature loop

Because the oven air temperature and the FPGA die temperature differ (the die generates heat internally), a slow outer loop adjusts the oven setpoint to drive the die temperature to the target. Every 1800 ticks (approximately 30 minutes) the sequencer:

1. Computes $e = T_{die,target} - T_{die,measured}$.
2. If $|e| > 3^\circ\text{C}$ (dead-band), shifts the oven setpoint by $\pm 1^\circ\text{C}$ in the direction that reduces the error.

This ensures that the FPGA transistors experience the intended stress temperature regardless of self-heating variations.

8. Setting Up a Fresh Workstation

8.1 System requirements

- OS: Ubuntu 22.04 LTS or Linux Mint 21+ (recommended). Other Debian-based distros should work with minor adjustments.
- Python: 3.10 or later (system package).
- Vivado: 2025.2 or later (required for bitstream builds and App_2Nexys auto-programming; not needed just to run App_Nexys with a pre-built bitstream).
- Disk space: ~70 GB for Vivado; ~500 MB for the repo + venv.

8.2 Step 1 — Install system dependencies

Listing 8.1: System package installation

```
sudo apt update
sudo apt install -y \
    python3 python3-pip python3-venv \
    libusb-1.0-0 libusb-dev \
    git
```

8.3 Step 2 — Clone the repository

```
git clone https://github.com/your-org/IC-Aging-Environment.git
cd IC-Aging-Environment
```

8.4 Step 3 — Install Vivado (if needed)

Download the Vivado 2025.2 installer from the Xilinx/AMD downloads page. Run the installer and select at minimum:

- **Vivado** (not just Vitis).
- **Artix-7** device support (under 7 Series).
- **Cable Drivers** (Digilent JTAG over USB).

Add Vivado to PATH in ~/.bashrc:

```
# Replace the path with your actual Vivado installation directory
echo 'source /opt/Xilinx/Vivado/2025.2/settings64.sh' >> ~/.bashrc
source ~/.bashrc
vivado -version # should print version info
```

Note

If you use App_2Nexys, you must also update VIVADO_BIN in App_2Nexys/config.py to point to your local Vivado binary. This path is currently hardcoded to a specific machine in the lab.

8.5 Step 4 — Configure udev rules for serial and JTAG devices

By default, non-root users cannot open USB serial ports or JTAG adapters. Two steps are needed:

8.5.1 Add user to the dialout group

```
sudo usermod -aG dialout $USER
# Log out and back in for the group change to take effect.
# Verify:
groups | grep dialout
```

8.5.2 Install Digilent USB cable rules

The Nexys4 DDR uses an FTDI chip programmed with Digilent’s VID/PID. Vivado ships the required udev rules:

```
# Path may differ; look for install_drivers in your Vivado
  installation
sudo
  /opt/Xilinx/Vivado/2025.2/data/xicom/cable_drivers/lin64/install_script/\
  install_drivers/install_drivers
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Warning

Without these rules, Vivado cannot program the FPGA via JTAG, and the serial port may not appear correctly. If you see **Permission denied** when opening a serial port, check group membership first.

8.6 Step 5 — Identify serial port assignments

Each Nexys4 DDR enumerates *two* `ttyUSB*` ports via its FTDI chip:

- **Lower-numbered port** (e.g., `ttyUSB0`): JTAG channel. Do not open this.
- **Higher-numbered port** (e.g., `ttyUSB1`): UART data channel. Use this for the DUT.

To identify which port is which:

```
# List USB serial devices with their USB IDs
ls -la /dev/serial/by-id/
# or use udevadm for details
udevadm info -a /dev/ttyUSB1 | grep -E "idVendor|idProduct|serial"
```

The Arduino will appear as a separate entry (typically an ACM or CH340 device).

8.7 Step 6 — Identify the PSU VISA resource

```
python3 -c "
import pyvisa
rm = pyvisa.ResourceManager('@py')
print(rm.list_resources())
"
# Expected output contains something like:
# ('USB0::0x1AB1::0x0E11::IT6502D300004::INSTR',)
```

Copy the full USB string; you will paste it into the Setup Dialog.

8.8 Step 7 — First run

```
# From repo root
./run.sh
```

On first launch, `run.sh` creates the Python virtual environment (`App_Nexys/.venv`) and installs all dependencies from `requirements.txt`. This may take a minute. Subsequent launches are instant.

The application opens. If `settings.json` does not exist, the Setup Dialog appears automatically.

9. Running an Experiment

9.1 Pre-experiment checklist

Before starting a test, verify:

- ☐ FPGA board is inside the oven with USB cable routed out.
- ☐ PSU is connected to the FPGA's power header; output is OFF.
- ☐ Arduino is connected via USB; serial cable to SSR is connected.
- ☐ SSR wiring to oven is correct (verify polarity and fusing).
- ☐ NTC thermistor is positioned inside the oven.
- ☐ All USB devices are enumerated on the PC (`lsusb`).

9.2 Setup Dialog

On first launch (or after clicking **Setup** in the menu), the Setup Dialog appears. Fill in:

Field	Value
DUT Port	Higher-numbered ttyUSB for the Nexys4 DDR (e.g., <code>/dev/ttyUSB1</code>)
DUT Baud	9600 (fixed; do not change)
Arduino Port	The Arduino's serial port (e.g., <code>/dev/ttyUSB2</code> or <code>/dev/ttyACM0</code>)
Enable Arduino	Check if the oven is connected
PSU Resource	The USB VISA string discovered in Step 6
Enable PSU	Check if the IT6502D is connected
VCCINT Setpoint	Target core voltage (V); default 1.0 V; raise to accelerate aging

Click **Save**. The settings are written to `settings.json`.

9.3 Setting experiment parameters

In the main window top bar:

1. **Oven setpoint:** type the desired temperature in the oven control group (e.g., 125 °C). The PID ramp will bring the oven there at 1 °C/s.
2. **Test name:** enter a descriptive name for the experiment. This becomes the CSV filename prefix.
3. **Enable Arduino / Enable PSU:** toggle these to match your hardware.

9.4 Starting the test

Click **Start Test**. The sequencer:

1. Enables PSU output and ramps to `VCCINT_SETPOINT_V`.
2. Sends the oven setpoint to the Arduino.
3. Begins polling the FPGA at 1 Hz.
4. Opens a new CSV file with a timestamped name.

The **Sensor** tab shows the live `dut_slack` value. The **Temperature** tab shows oven and DUT temperatures.

9.5 Monitoring

Experiments run unattended for days. The app continues logging even if the display goes to sleep. Recommended practice:

- Disable screen lock and display power management on the test machine.
- Check in on the CSV daily to verify that new rows are being written.
- Monitor `dut_temp_c` in the log: it should be stable within $\pm 1^\circ\text{C}$ of the target once the PID has settled (approximately 30 min after start).

9.6 Safety limits

The TestSequencer monitors safety limits every tick. If any limit is exceeded, the test is automatically stopped and the PSU is disabled:

- Oven temperature $> 130^\circ\text{C}$
- DUT die temperature $> 160^\circ\text{C}$
- PSU current $> 3.0\text{ A}$

An event is logged to the on-screen log panel with a timestamp.

9.7 Stopping the test

Click **Stop Test**. The sequencer:

1. Closes the CSV file and writes an end-of-test footer.
2. Disables PSU output.
3. Sends a 0°C setpoint to the Arduino (which effectively stops heating).

Allow the oven to cool before opening it. Do not disconnect USB cables while the oven is hot.

10. Data Analysis

10.1 CSV file structure

Log files are in `App_Nexys/test_logs/`. Each file begins with comment lines (`#`) containing metadata, followed by a header row, then data rows at 1-second intervals.

10.2 Loading data in Python (Pandas)

Listing 10.1: Loading and plotting `dut_slack` over time

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('test_logs/MyTest_20260101_120000.csv', comment='#')

fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True)

axes[0].plot(df['time_sec'] / 3600, df['dut_slack'])
axes[0].set_ylabel('Slack (phase steps)')
axes[0].set_title('Timing Margin vs. Time')

axes[1].plot(df['time_sec'] / 3600, df['dut_temp_c'], label='DUT die')
axes[1].plot(df['time_sec'] / 3600, df['oven_temp_c'], label='Oven air',
             alpha=0.5)
axes[1].set_ylabel('Temperature (degC)')
axes[1].legend()

axes[2].plot(df['time_sec'] / 3600, df['dut_volt'])
axes[2].set_ylabel('VCCINT (V)')
axes[2].set_xlabel('Time (hours)')

plt.tight_layout()
plt.savefig('aging_result.png', dpi=150)
plt.show()
```

10.3 Typical result signature

A successful aging run shows:

- `dut_slack` decreasing monotonically over the stress period.
- `dut_temp_c` stable within $\pm 1^\circ\text{C}$ of the target after the initial ramp.
- `dut_volt` stable within $\pm 5\text{ mV}$ of the setpoint.
- `error_count` (in the raw packet; not in the default CSV) remaining at 0 until functional failure begins.

If `dut_slack` shows large random jumps, check:

1. Temperature stability — thermal gradients affect timing.
2. V_{CCINT} stability — supply noise directly modulates path delay.
3. USB cable quality — occasional packet errors produce spurious readings.

10.4 Fitting the degradation model

To extract the aging exponent n from the NBTI power-law:

Listing 10.2: Fitting slack vs. time to a power-law model

```
import numpy as np
from scipy.optimize import curve_fit

# Use only data after thermal stabilisation (e.g., skip first 3600 s)
mask = df['time_sec'] > 3600
t = df.loc[mask, 'time_sec'].values
s = df.loc[mask, 'dut_slack'].values

# Model: slack(t) = s0 - A * t^n
def model(t, s0, A, n):
    return s0 - A * t**n

p0 = [s.max(), 1.0, 0.15]
popt, _ = curve_fit(model, t, s, p0=p0, maxfev=10000)
s0, A, n = popt
print(f"Initial slack: {s0:.1f}   A: {A:.4f}   n: {n:.4f}")
```

The fitted exponent n can be compared against theoretical NBTI predictions ($n \approx 0.1$ – 0.25).

11. Other Applications

11.1 App_2Nexys (dual DUT)

App_2Nexys operates two Nexys4 DDR boards simultaneously with a shared oven, two PSUs (IT6502D + Agilent E3634A), and two independent DUT workers. Both FPGAs are programmed automatically at test start via a Vivado batch session.

Key differences from App_Nexys:

- The DUT outer temperature loop uses the *average* of both DUT temperatures.
- Port pairing is done by fixed USB serial IDs (USB_ID_DUT0/DUT1).
- VIVADO_BIN is hardcoded in App_2Nexys/config.py — update this when deploying on a new machine.
- The 15-byte DUT protocol is identical to App_Nexys.

11.2 App_FPGAging_Slack_Sensor (UltraScale+ / SBCCI target)

This application targets an UltraScale+ FPGA accessed via an ESP32 UART router and an STM32 bridge. It supports a different 9-byte packet format from the CROC FPGA and uses CRC16-Modbus framing for STM32 communication. This is an advanced configuration used for the SBCCI publication target.

12. Troubleshooting

12.1 Serial port not accessible

Symptom: Permission denied: `'/dev/ttyUSB1'` or port not found.

Fix:

```
# Verify group membership
groups | grep dialout
# If dialout is not listed:
sudo usermod -aG dialout $USER
# Then log out and log back in.
```

12.2 Wrong ttyUSB port selected (JTAG vs UART)

Symptom: App connects but no data arrives; DUT field stays at zero.

Fix: The lower-numbered port of a Nexys4 DDR is JTAG, not UART. Always select the *higher-numbered* of the two ports in the Setup Dialog. Use `ls -la /dev/serial/by-id/` to inspect the symlinks and identify port numbers.

12.3 App_2Nexys fails to program FPGAs

Symptom: Test starts but the FSM logs “Vivado batch job failed”.

Fix: `VIVADO_BIN` in `App_2Nexys/config.py` is hardcoded to an absolute path from another machine. Update it to the local Vivado installation:

```
# Find the vivado binary
which vivado      # if already on PATH
# or
find /opt -name vivado -type f 2>/dev/null | head -5

# Then edit App_2Nexys/config.py:
# VIVADO_BIN = '/opt/Xilinx/Vivado/2025.2/bin/vivado'
```

12.4 PSU VISA string error

Symptom: `config.load_config()` silently clears the PSU port; the PSU shows as “not configured” even though a port was saved.

Cause: The saved port string does not start with USB. `load_config()` rejects non-USB strings to prevent the IT6502D from being opened as a serial device.

Fix: Re-run the resource discovery command and paste the full USB string into the Setup Dialog.

12.5 dut_slack shows erratic jumps

Cause: Most commonly thermal or voltage instability during the initial warm-up.

Fix:

1. Wait at least 30 minutes after pressing Start before drawing conclusions.
2. Verify `dut_temp_c` has stabilised.
3. Verify `dut_volt` is within 5 mV of the setpoint.
4. If the XADC is not yet settled after power-on, readings of $> 200^{\circ}\text{C}$ will be silently discarded by `DUTWorker`.

12.6 Vivado timing reports show negative slack on the sensor path

This is expected and intentional. The sensor path is deliberately placed at its timing limit. Vivado's timing analysis uses post-P&R RC delays that may not exactly match the actual operating condition. The MMCM phase measurement accounts for the real in-situ timing directly.

A. FOPDT Step Test Procedure

The FOPDT model of Equation 7.1 was identified from a step-test experiment. To repeat this experiment (e.g., after changing the oven or SSR):

1. Flash `Arduino-ESP/FOPDT_Step_Test.ino` to the Arduino.
2. Open a serial monitor at 115200 baud.
3. Wait for the oven to reach thermal steady state at a low output (e.g., 20%).
4. Apply a step change to a higher output (e.g., 40%).
5. Record temperature vs. time until a new steady state is reached.
6. Fit the FOPDT model to extract K , θ , and τ .
7. Apply SIMC tuning rules to compute new PID gains.
8. Update `PID_KP`, `PID_KI`, `PID_KD` in `App_Nexys/config.py`.

B. Regenerating Fixed PnR Constraints

The `fixed_pnr_constraints.xdc` file locks all sensor primitives to specific BEL sites. Regenerate it only after a deliberate new place-and-route run:

```
cd vivado/aging_study_nexys4ddr

# Build a new bitstream AND update the reference checkpoint +
# constraints
scripts/build_bitstream.sh --jobs 8 --refresh-ref

# Or, if you already have a fresh checkpoint and just want to
# regenerate XDC:
scripts/extract_fixed_pnr_constraints.sh
```

After regeneration, commit both `fixed_pnr_constraints.xdc` and `references/fixed_pnr.dcp`. All previously collected timing data is now incomparable with future data — document the PnR change in the experiment log.